

Build Task — HTTP Client Wrapper — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (how a real HTTP client wrapper is actually written, including what makes it satisfy this Build Task's testability requirement). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building one function that other code will call instead of calling `requests` directly: something like `request_with_retry(method, url, ...)`. Its whole job is to hide all the messy failure-handling from this document's exercises behind one clean call.

Inside, it needs to: make the call with a timeout, decide whether a failure is worth retrying, wait longer between each retry, give up after a limit, and turn the final result into either a parsed dictionary or a clear, typed error.

Things to use:

- `requests.request(method, url, timeout=..., **kwargs)` — one function that handles GET/POST/etc. based on the `method` string.
- `response.status_code // 100` — dividing by 100 gives you the status category (2, 4, or 5) without writing out every code.
- `random.uniform(0, 1)` — adds jitter (a small random extra wait) on top of your backoff delay.
- The logger from `01_python_foundations`'s `logging_setup.py` — `get_logger(__name__)`, then `.debug(...)` per attempt, `.error(...)` on final failure.
- Your two new error classes go in `exceptions.py`, next to Doc01's `MissingConfigError` — keep that one in the file, because the copied `config.py` imports it from there.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You're combining everything from this document's exercises into one function:

```
request_with_retry(method: str, url: str, **kwargs) -> dict .
```

The core design decision is a **classification step**: every failure needs to be sorted into "transient" (worth retrying: timeout, 5xx, 429) or "permanent" (not worth retrying: any other 4xx). This classification should probably be its own small piece of logic, separate from the retry loop itself, so you can test it on its own.

The retry loop then wraps a single attempt, using your `compute_backoff_delay(attempt)` helper from the timeout/retry exercise, and stops either on success or once a permanent failure is classified (fail immediately, no retry) or the attempt limit is hit (raise `TransientHTTPError`).

Here's what to actually go look at:

- `requests.request(method, url, timeout=..., **kwargs)` — the generic entry point `requests.get` / `.post` /etc. all call underneath; taking `method` as a string parameter lets your one function handle every HTTP verb.
- **Classifying by status code** — `response.status_code == 429` is transient. `response.status_code // 100 == 5` is transient. `response.status_code // 100 == 4` (and not 429) is permanent. Anything else (2xx, 3xx) is success.
- `requests.exceptions.Timeout` and `requests.exceptions.ConnectionError` — both count as transient failures, same bucket as a 5xx.
- `compute_backoff_delay(attempt) + random.uniform(0, jitter_max)` — jitter is just adding a small random number on top of your existing exponential formula.
- **Attaching the response body to `PermanentHTTPError`** — store it as an attribute (`self.body = response.text`) in `__init__`, so a caller catching the error can actually see what the server said was wrong.

Sketch the function's overall shape — the loop, the classification call, and the two error types it can raise — before checking Hint 2.

Think past "classify the failure and retry it correctly." Ask: **this function is supposed to be testable without real network calls where possible, per the Build Task's own constraint — which parts of it can actually be tested that way, and how did the earlier exercises already show you?**

The answer isn't a new trick — it's the same move every exercise in this document already made: pull the *decision* logic out into its own small, pure function, and test that directly with plain values.

`is_transient_status(status_code)` above is one such function.

`compute_backoff_delay(attempt)` from the timeout/retry exercise is another. The rate-limit

exercise's `decide_wait_seconds(headers, attempt)` is a third — it takes a plain `headers` dict, not a real response object, so it's testable the exact same way. The retry loop itself — the part that actually calls `requests.request` — gets exercised with a real call, the same way the timeout/retry exercise's `get_with_retry()` was: no fake server, nothing new to learn.

There's a second question worth asking too: should every call to `request_with_retry()` open a brand-new connection, or should repeated calls to the same host reuse one, the way the session-reuse exercise showed? A module-level `requests.Session()`, reused across calls, is both faster (connection pooling) and the natural place to set default headers once (like a User-Agent identifying your client) instead of on every call.

The extra pieces:

- A module-level `requests.Session()` in `http_client.py`, called as `_session.request(...)` inside the loop instead of the bare `requests.request`.
- `decide_wait_seconds(headers, attempt)` from the rate-limit exercise, called inside the retry loop before falling back to `compute_backoff_delay(attempt)` — a 429 should respect `Retry-After` if the server sent one, the same as any other real client.

Difference between Basic and Intermediate: Basic names the function and the tools. Intermediate designs the classification step and the loop's shape around it, then applies the same "pull the decision logic into its own pure function" move from the timeout/retry and rate-limit exercises to everything in this function that can be tested without a network call, and folds in the session-reuse exercise's connection pooling — because a real `http_client.py` isn't graded on one exercise's requirements in isolation, it's the one file every later document actually imports.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
exceptions.py:
    keep Doc01's MissingConfigError exactly as it was
    make TransientHTTPError, a kind of Exception
    make PermanentHTTPError, a kind of Exception, stores the response body

http_client.py:
    function compute_backoff_delay(attempt):
        return 2 to the power of attempt, plus a small random amount

    function request_with_retry(method, url, max_attempts=5, **kwargs):
        for attempt from 0 to max_attempts - 1:
            log "attempt N" at DEBUG level
            try:
                call requests.request with a timeout
            except timeout or connection error:
                if this was the last attempt:
                    log ERROR, raise TransientHTTPError
                else: wait, try again

            if status is 2xx: return the parsed JSON
            if status is 429 or 5xx:
                if this was the last attempt:
                    log ERROR, raise TransientHTTPError
                else: wait, try again
            if status is any other 4xx:
                log ERROR, raise PermanentHTTPError right away, no retry
```

The trickiest part — turning a bad JSON body into a clear error instead of a raw crash:

```
import json

def parse_json_or_raise(response):
    try:
        return response.json()
```

```
except json.JSONDecodeError as e:  
    raise TransientHTTPError("response was not valid JSON: " + str(e))
```

Call this instead of `response.json()` directly inside `request_with_retry`. Try finishing the rest yourself before checking the Intermediate Version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with the classification split into its own function

exceptions.py :

```
class MissingConfigError is an Exception    (kept from Doc01)
class TransientHTTPError is an Exception
class PermanentHTTPError is an Exception, holds status_code and body
```

http_client.py :

```
function compute_backoff_delay(attempt) -> float:
    base = 2 ** attempt
    jitter = a random float between 0 and 1
    return base + jitter

function is_transient_status(status_code) -> bool:
    return status_code == 429 or status_code // 100 == 5

function request_with_retry(method, url, max_attempts=5, **kwargs) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(method, url, timeout=(3, 10), **kwargs)
        except (Timeout, ConnectionError) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            sleep(compute_backoff_delay(attempt))
            continue

    if response.status_code < 300:
        try:
            return response.json()
        except json.JSONDecodeError as e:
            raise TransientHTTPError(f"bad JSON in response: {e}") from e

    if is_transient_status(response.status_code):
        if attempt == max_attempts - 1:
            logger.error(
```

```

        f"gave up after {max_attempts} attempts, "
        f"last status {response.status_code}"
    )
    raise TransientHTTPError(
        f"status {response.status_code} "
        f"after {max_attempts} attempts"
    )
    sleep(compute_backoff_delay(attempt))
    continue

# any other 4xx: permanent, no retry
logger.error(f"permanent failure: status {response.status_code}")
raise PermanentHTTPError(response.status_code, response.text)

```

The key structural idea: the loop has exactly two ways out — a successful return, or a raised error — and exactly one way to keep going — `continue` after sleeping. Nothing falls through silently. Write out the full, typed version yourself, then go one step further — Approach 2, which folds in the session-reuse and rate-limit exercises.

Approach 2 — a shared session and `Retry-After` support

```

http_client.py:
    module-level: session = requests.Session()    # reused across every call

    function request_with_retry(method, url, max_attempts=5, **kwargs) -> dict:
        for attempt in range(max_attempts):
            logger.debug(f"attempt {attempt}: {method} {url}")
            try:
                response = session.request(
                    method, url, timeout=(3, 10), **kwargs
                )
            except (Timeout, ConnectionError) as e:
                if attempt == max_attempts - 1:
                    raise TransientHTTPError(...) from e
                sleep(compute_backoff_delay(attempt))
                continue

            if response.status_code < 300:
                return parse_json_or_raise(response)

            if is_transient_status(response.status_code):
                if attempt == max_attempts - 1:
                    raise TransientHTTPError(...)
                # respects Retry-After if the server sent one
                wait = decide_wait_seconds(response.headers, attempt)
                sleep(wait)

```

```
continue
```

```
raise PermanentHTTPError(response.status_code, response.text)
```

Here's almost the whole thing — fill in the missing piece yourself:

```
import json
import random
import time
from typing import Any

import requests

from exceptions import PermanentHTTPError, TransientHTTPError
from logging_setup import get_logger

logger = get_logger(__name__)
_session = requests.Session()

def compute_backoff_delay(attempt: int) -> float:
    return (2 ** attempt) + random.uniform(0, 1)

def is_transient_status(status_code: int) -> bool:
    return status_code == 429 or status_code // 100 == 5

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    **kwargs: Any,
) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = _session.request(method, url, timeout=(3, 10), **kwargs)
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
```

```

        continue

    if response.status_code < 300:
        try:
            return response.json()
        except json.JSONDecodeError as e:
            raise TransientHTTPError(f"bad JSON in response: {e}") from e

    if is_transient_status(response.status_code):
        # your turn: on the last attempt, log ERROR and raise
        # TransientHTTPError; otherwise sleep and continue – for the
        # sleep, reuse decide_wait_seconds(headers, attempt) from the
        # rate-limit exercise so a real Retry-After header is respected
        ...

    logger.error(f"permanent failure: status {response.status_code}")
    raise PermanentHTTPError(response.status_code, response.text)

```

Fill in the transient branch yourself, then compare all of your finished versions against the [Solution](#).

Difference between Basic and Intermediate: Basic's pseudocode calls `requests.request` by name and never respects `Retry-After`. Intermediate Approach 1 is the same shape, fully typed, with the transient/permanent split as its own function. Approach 2 reuses a module-level `Session` for connection pooling and folds in `Retry-After` handling from the rate-limit exercise — the combination this document's 5 exercises were always building toward.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Every code block below shows what you'd see running it against a real, working call unless a block says otherwise. Read both depths — they're not "wrong, right," they're 2 real, valid stages of building the same client, with real tradeoffs between them.

Basic Version

Approach 1 — one function, everything inline

Story — `exceptions.py` : `request_with_retry()` needs a way to tell its caller "this is worth retrying elsewhere too" apart from "this will never work, stop trying" — two error types make that distinction part of the function's actual contract, not just a comment. **If not:** every later document that imports `http_client.py` would catch one generic exception and have no way to tell a transient network hiccup from its own bad request.

```
# exceptions.py
# kept from Doc01, unchanged – config.py imports it from this file
# Why: gives load_config() its own error type, so calling code can catch a
# missing
# setting specifically, instead of catching every possible Exception blindly.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass

# Why: two separate types so callers can catch "worth retrying" and
# "my mistake, don't retry" differently, instead of one generic Exception.
class TransientHTTPError(Exception):
    pass

class PermanentHTTPError(Exception):
    def __init__(self, status_code, body):
        self.status_code = status_code
        self.body = body
        super().__init__("HTTP " + str(status_code) + ": " + body)
```

Story — `http_client.py` : this is the Goal of the whole Build Task — one wrapper around `requests` that every later document imports instead of calling the network directly, so

timeout/retry/classification logic exists exactly once in the whole curriculum. **If not:** every later document (Doc04's `chat_client.py`, and beyond) would re-implement its own retry loop, each slightly different, each with its own undiscovered bugs.

```
# http_client.py
import json
import random
import time
import requests
from exceptions import TransientHTTPError, PermanentHTTPError
from logging_setup import get_logger

logger = get_logger(__name__)

def compute_backoff_delay(attempt):
    # Why: spaces retries out (2, 4, 8... seconds) with a little randomness,
    # so a burst of clients retrying together doesn't all hit the server at
    # once.
    return (2 ** attempt) + random.uniform(0, 1)

def request_with_retry(method, url, max_attempts=5, **kwargs):
    # Why: the one function every later document calls instead of requests
    # directly – hides the timeout/retry/classify logic behind one clean call.
    for attempt in range(max_attempts):
        logger.debug("attempt " + str(attempt) + ": " + method + " " + url)
        try:
            response = requests.request(method, url, timeout=(3, 10), **kwargs)
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(
                    "gave up after " + str(max_attempts)
                    + " attempts: " + str(e)
                )
                raise TransientHTTPError(str(e))
            time.sleep(compute_backoff_delay(attempt))
            continue

    if response.status_code < 300:
        try:
            return response.json()
        except json.JSONDecodeError as e:
            raise TransientHTTPError("bad JSON in response: " + str(e))
```

```

    if response.status_code == 429 or response.status_code // 100 == 5:
        if attempt == max_attempts - 1:
            logger.error(
                "gave up, last status " + str(response.status_code)
            )
            raise TransientHTTPError("status " + str(response.status_code))
            time.sleep(compute_backoff_delay(attempt))
            continue

        logger.error("permanent failure: status " + str(response.status_code))
        raise PermanentHTTPError(response.status_code, response.text)

if __name__ == "__main__":
    print(request_with_retry("GET", "https://api.github.com"))

```

Expected output:

```
{'current_user_url': 'https://api.github.com/user', ...}
```

This version works correctly and meets every Build Task requirement. It's missing type hints, doesn't separate the transient/permanent decision into its own testable function, and calls `requests.request` directly, which makes it awkward to test without a real network call — all fine for a first working version.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — everything typed, still inline

exceptions.py

```
# kept from Doc01, unchanged – config.py imports it from this file
# Why: gives load_config() its own error type, so calling code can catch a
# missing
# setting specifically, instead of catching every possible Exception blindly.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass

class TransientHTTPError(Exception):
    """Raised when a retryable failure (timeout, 5xx, 429) exhausts all
    attempts."""
    pass

class PermanentHTTPError(Exception):
    """Raised immediately for a non-retryable 4xx, with the response body
    attached."""

    def __init__(self, status_code: int, body: str) -> None:
        self.status_code = status_code
        self.body = body
        super().__init__(f"HTTP {status_code}: {body[:200]}")
```

http_client.py

```
import json
import random
import time
from typing import Any

import requests

from exceptions import PermanentHTTPError, TransientHTTPError
from logging_setup import get_logger
```

```

logger = get_logger(__name__)

DEFAULT_TIMEOUT = (3, 10)

def compute_backoff_delay(attempt: int) -> float:
    return (2 ** attempt) + random.uniform(0, 1)

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    **kwargs: Any,
) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(
                method, url, timeout=DEFAULT_TIMEOUT, **kwargs
            )
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
            continue

        if response.status_code < 300:
            try:
                return response.json()
            except json.JSONDecodeError as e:
                raise TransientHTTPError(f"bad JSON in response: {e}") from e

        if response.status_code == 429 or response.status_code // 100 == 5:
            if attempt == max_attempts - 1:
                logger.error(f"gave up, last status {response.status_code}")
                raise TransientHTTPError(
                    f"status {response.status_code} "
                    f"after {max_attempts} attempts"
                )
            time.sleep(compute_backoff_delay(attempt))
            continue

```

```

        logger.error(f"permanent failure: status {response.status_code}")
        raise PermanentHTTPError(response.status_code, response.text)

if __name__ == "__main__":
    print(request_with_retry("GET", "https://api.github.com"))

```

Why this approach: everything lives in one place, which is easy to read start to finish for a project this size.

Approach 2 — the transient/permanent decision pulled into its own testable function

```

def is_transient_status(status_code: int) -> bool:
    """A 429, or any 5xx, is worth retrying. Everything else that isn't a
    success is a permanent failure the caller needs to fix, not retry."""
    return status_code == 429 or status_code // 100 == 5

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    **kwargs: Any,
) -> dict:
    for attempt in range(max_attempts):
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            response = requests.request(
                method, url, timeout=DEFAULT_TIMEOUT, **kwargs
            )
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,
        ) as e:
            if attempt == max_attempts - 1:
                logger.error(f"gave up after {max_attempts} attempts: {e}")
                raise TransientHTTPError(str(e)) from e
            time.sleep(compute_backoff_delay(attempt))
            continue

    if response.status_code < 300:
        try:
            return response.json()
        except json.JSONDecodeError as e:
            raise TransientHTTPError(f"bad JSON in response: {e}") from e

```

```

if is_transient_status(response.status_code):
    if attempt == max_attempts - 1:
        logger.error(f"gave up, last status {response.status_code}")
        raise TransientHTTPError(
            f"status {response.status_code} "
            f"after {max_attempts} attempts"
        )
    time.sleep(compute_backoff_delay(attempt))
    continue

logger.error(f"permanent failure: status {response.status_code}")
raise PermanentHTTPError(response.status_code, response.text)

```

Why this approach: `is_transient_status(429)` and `is_transient_status(404)` can be tested directly with plain integers — no fake `Response` object, no mocking `requests` at all, the same as `compute_backoff_delay()` in the timeout/retry exercise. The loop itself (`request_with_retry`) still calls `requests.request` for real — that part gets tested by actually calling it, the same way the timeout/retry exercise's `get_with_retry()` was.

Difference from Basic: both Intermediate approaches add full type hints and a `DEFAULT_TIMEOUT` constant instead of a repeated magic tuple. Approach 2 additionally separates the transient/permanent decision into its own pure function, testable in complete isolation from the network.

Approach 3 — a shared session, and `Retry-After` support

```

# exceptions.py
# kept from Doc01, unchanged – config.py imports it from this file
# Why: gives load_config() its own error type, so calling code can catch a
# missing
# setting specifically, instead of catching every possible Exception blindly.
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass

# Why: two separate types so callers can catch "worth retrying" and
# "my mistake, don't retry" differently, instead of one generic Exception.
class TransientHTTPError(Exception):
    """Raised when a retryable failure (timeout, 5xx, 429) exhausts all
    attempts."""
    pass

class PermanentHTTPError(Exception):

```

```
"""Raised immediately for a non-retryable 4xx, with the response body
attached."""
```

```
def __init__(self, status_code: int, body: str) -> None:
    self.status_code = status_code
    self.body = body
    super().__init__(f"HTTP {status_code}: {body[:200]}")
```

```
# http_client.py
import json
import random
import time
from datetime import datetime, timezone
from email.utils import parsedate_to_datetime
from typing import Any

import requests

from exceptions import PermanentHTTPError, TransientHTTPError
from logging_setup import get_logger

logger = get_logger(__name__)

DEFAULT_TIMEOUT = (3, 10)
MAX_RETRY_AFTER_SECONDS = 60.0

_session = requests.Session()
_session.headers.update({"User-Agent": "learning-langgraph-http-client/1.0"})

def compute_backoff_delay(attempt: int) -> float:
    # Why: spaces retries out (2, 4, 8... seconds) with a little randomness,
    # so a burst of clients retrying together doesn't all hit the server at
    # once.
    return (2 ** attempt) + random.uniform(0, 1)

def is_transient_status(status_code: int) -> bool:
    # Why: pulled out on its own so it's testable with plain integers –
    # no fake Response object, no network, no mocking needed.
    return status_code == 429 or status_code // 100 == 5

def parse_retry_after(value):
    # Why: the HTTP spec allows Retry-After to be a plain number of seconds
    # OR an HTTP-date string – this handles both instead of crashing on one.
```

```

# Same function as the rate-limit exercise, copied in unchanged.
try:
    return float(value)
except ValueError:
    pass
try:
    target_time = parsedate_to_datetime(value)
    now = datetime.now(timezone.utc)
    seconds_left = (target_time - now).total_seconds()
    return max(seconds_left, 0.0)
except (TypeError, ValueError):
    return None

def decide_wait_seconds(headers: dict, attempt: int) -> float:
    # Why: respects a real Retry-After header when the server sends one,
    # and falls back to exponential backoff (capped) when it doesn't. Takes
    # a plain headers dict, not a Response object, so it's testable with
    # plain values – same signature as the rate-limit exercise.
    retry_after = headers.get("Retry-After")
    wait = None
    if retry_after is not None:
        wait = parse_retry_after(retry_after)
    if wait is None:
        wait = compute_backoff_delay(attempt)
    return min(wait, MAX_RETRY_AFTER_SECONDS)

def request_with_retry(
    method: str,
    url: str,
    max_attempts: int = 5,
    **kwargs: Any,
) -> dict:
    # why: the one function every later document imports instead of calling
    # requests directly.
    for attempt in range(max_attempts):
        # how: DEBUG, not print – this is diagnostic, not output
        logger.debug(f"attempt {attempt}: {method} {url}")
        try:
            # how: reuses the module-level Session for connection pooling
            response = _session.request(
                method, url, timeout=DEFAULT_TIMEOUT, **kwargs
            )
        except (
            requests.exceptions.Timeout,
            requests.exceptions.ConnectionError,

```

```

    ) as e:
        # when: the request itself never got a response at all –
        # treated as transient, same bucket as a 5xx status.
        if attempt == max_attempts - 1:
            logger.error(f"gave up after {max_attempts} attempts: {e}")
            raise TransientHTTPError(str(e)) from e
        time.sleep(compute_backoff_delay(attempt))
        # how: goes back to the top of the loop for the next attempt
        continue

    if response.status_code < 300:
        # when: 2xx/3xx is success – try to parse it and return.
        try:
            return response.json()
        except json.JSONDecodeError as e:
            # why: a 200 with an unparseable body is still a real
            # failure the caller needs to know about, not a silent None.
            raise TransientHTTPError(f"bad JSON in response: {e}") from e

    if is_transient_status(response.status_code):
        # when: 429 or 5xx – worth retrying, unless this was the last
        # attempt.
        if attempt == max_attempts - 1:
            logger.error(f"gave up, last status {response.status_code}")
            raise TransientHTTPError(
                f"status {response.status_code} "
                f"after {max_attempts} attempts"
            )
        # how: honors Retry-After if the server sent one
        time.sleep(decide_wait_seconds(response.headers, attempt))
        continue

    # when: any other 4xx – this is a client mistake that will fail
    # again the same way, so raise immediately instead of retrying.
    logger.error(f"permanent failure: status {response.status_code}")
    raise PermanentHTTPError(response.status_code, response.text)

if __name__ == "__main__":
    print(request_with_retry("GET", "https://api.github.com"))

```

Expected output:

```
{'current_user_url': 'https://api.github.com/user', ...}
```

Story — `test_http_client.py`: `http_client.py` is about to get imported by every later document — Doc04's `chat_client.py` calls through it for every OpenAI request. A regression in the classification or backoff math (a 404 gets retried when it shouldn't, `Retry-After` stops being respected) needs to be caught right here, in milliseconds, the same way the timeout/retry and rate-limit exercises already caught theirs. **If not:** the only way to know the decision logic still works would be to read the code carefully every time, instead of just running it.

This test file uses the exact same style as every exercise you've already worked through in this document: call the function directly, `print()` the result, and compare it against a `# expected value` comment right next to the call. Nothing here is new — no test framework, no fake response objects standing in for the network. `is_transient_status`, `compute_backoff_delay`, and `decide_wait_seconds` are all plain functions that take plain values (an `int`, a `dict`), so they're testable exactly like `compute_backoff_delay()` was in the timeout/retry exercise and `decide_wait_seconds()` was in the rate-limit exercise. The retry loop itself — the part that actually calls the network — is exercised with one real call at the bottom, the same way `get_with_retry()` was tested in the timeout/retry exercise.

How to run it: no install needed — it's a plain Python script. From inside `practice/build_task/`, with `http_client.py` and `exceptions.py` sitting right next to it:

```
python test_http_client.py
```

`test_http_client.py`:

```
from http_client import (
    compute_backoff_delay,
    decide_wait_seconds,
    is_transient_status,
    request_with_retry,
)

# is_transient_status: worth retrying vs. a real mistake to fix
for code in (429, 500, 503, 404, 401):
    print(f"is_transient_status({code}) -> {is_transient_status(code)}")

# compute_backoff_delay: no network, no time.sleep, no mocking needed
delay = compute_backoff_delay(3)
print(f"compute_backoff_delay(3) -> {delay}")

# decide_wait_seconds: respects Retry-After, falls back to backoff, caps a
# huge value -- same three cases the rate-limit exercise already proved
told_to_wait = decide_wait_seconds({"Retry-After": "2"}, attempt=0)
print(f"decide_wait_seconds, server said wait 2s -> {told_to_wait}")

no_header = decide_wait_seconds({}, attempt=3)
```

```

print(f"decide_wait_seconds, no header, attempt 3 -> {no_header}")

huge_value = decide_wait_seconds({"Retry-After": "999999999"}, attempt=0)
print(f"decide_wait_seconds, server said wait 999999999s -> {huge_value}")

# the retry loop itself: one real call, same as the timeout/retry exercise
if __name__ == "__main__":
    result = request_with_retry("GET", "https://api.github.com")
    print(f"request_with_retry(\"GET\", api.github.com) -> {result}")

```

Expected output:

```

is_transient_status(429) -> True
is_transient_status(500) -> True
is_transient_status(503) -> True
is_transient_status(404) -> False
is_transient_status(401) -> False
compute_backoff_delay(3) -> 8.417...
decide_wait_seconds, server said wait 2s -> 2.0
decide_wait_seconds, no header, attempt 3 -> 8.417...
decide_wait_seconds, server said wait 999999999s -> 60.0
request_with_retry("GET", api.github.com) ->
{'current_user_url': 'https://api.github.com/user', ...}

```

Each line says what was called and what it returned, right next to each other — so checking your own run against this one is just reading down the list, not counting which bare `True` matches which call.

Difference from Approach 2: Approach 2's `request_with_retry` is already correct and already imports the session-reuse and rate-limit exercises' work. Approach 3 doesn't change the design — it's the same function, with `parse_retry_after`'s HTTP-date support and a capped `MAX_RETRY_AFTER_SECONDS` folded in from the rate-limit exercise's own Approach 2, plus `test_http_client.py` proving each pure piece directly.

Which one should you actually write? Approach 2 already meets every requirement. Approach 3 is worth it once you want the exact same `Retry-After` robustness the rate-limit exercise already built — the HTTP-date form, and a hard cap so a buggy or malicious server can't stall your client for years. Either way, keep the tests limited to the pure functions, exactly like every exercise before this one did — the retry loop itself is proven by actually calling it, not by building a fake network to avoid calling it.